

Cohort module six, two hours

Orchestrate and Ship Verified PRs

Everything connects. The last half hour is the build against buy conversation, answered by mapping what they built onto a hosted product.

0:00 to 0:45

One workflow, six layers

Goal Each person's pieces run as a single command against a real item in their own repository.

Say

- Resist rebuilding anything. This session connects what exists, and the connecting code should be short and boring.
- Walk the seams rather than the parts: what triage hands to implementation, what implementation hands to checking, what checking hands to delivery.
- Point out that every handoff is written somewhere durable, so a run that dies can be picked up rather than restarted.

Run

```
$ make demo STEP=06
```

Room does

- Runs the whole sequence on their own item, unattended.
- Reads their own handoff records and confirms each stage could be resumed.

Ask *Kill your run halfway. Can the next one continue from what it left behind?*

0:45 to 1:15

What a reviewer receives

Goal Their requests arrive with enough attached that a colleague could judge one in a few minutes.

- Say**
- Have them review each other's output rather than their own. Reviewing your own factory tells you nothing.
 - Collect what reviewers wanted and did not find, and add those to the template on the spot.
 - Be firm about the boundary. Nothing merges itself at any tier, and the enforcement belongs in the repository settings rather than in a script.
-

Run

```
$ node steps/06-delivery/demo.ts open-pr  
$ node steps/06-delivery/demo.ts try-merge
```

- Room does**
- Reviews a peer's request and reports what was missing.
 - Watches an attempted merge blocked, and checks their own protection settings.
-

Ask *What did your partner need that your template did not give them?*

1:15 to 1:45

The same thing, bought

Goal Everyone can decide for their own organisation, with reasons that are not about preference.

Say

- Open the hosted product and map its stages onto their own, out loud, one at a time. The correspondence is close and that is the lesson.
- Show where their policy lands on the other substrate: a function that permits or refuses a move, holding the same rules they wrote in week one.
- Frame the decision honestly. This is about who maintains the parts you now understand, not about whether to have them.

Run

```
$ make demo STEP=07
```

The mapping is written out in `steps/07-mastra/README.md`.

Room does

- Sees a board whose columns are the stages they built by hand.
- Sees their protected path expressed as a policy function on the other substrate.

Ask *Which layers would you hand to a vendor, and which would you never give up?*

1:45 to 2:00

Demonstration day

Goal	Everyone knows the shape of what they will show and how long they have.
Say	• Give the running order and the timing, and say what you will ask each of them. • Set the expectation that a refusal or an escalation is a better demonstration than a clean pass, because it shows the limits working.
Run	Nothing. Schedule on screen.
Room does	Picks a slot and confirms their item still runs end to end.
Ask	<i>Anyone whose item stopped working this week? Bring it to Friday instead of debugging alone.</i>

1:35 to 1:50

Finished, and still not merged

Goal They stop treating a board reaching done as a claim that anything shipped.

- Say**
- Done on the board means the machine has nothing left to do, which is not the same as merged.
 - Two boards, and this is the one for issues; the pull request lives on the other with its own phases.
 - Anyone who conflates them will report a week of work that is still sitting unreviewed.

Run

```
$ make e2e
```

The item reaches done while its pull request stays open.

Show on the board Review to its own phases to the same change on Work to done, unmerged

Done means the machine is finished

The work item reaches done when its pull request exists. Nothing has merged, nobody has read it, and the board is not claiming otherwise.

The same change, two boards

Find it on Work as an issue and on Review as a pull request. They have different phases because they are different questions: is this worth doing, and is this done well.

A re-review needs a push

The review board has no review-to-review step. Fixing the branch is what starts the second pass, and asking the board to review again does nothing at all.

Room does

- Finds the same change represented on both boards.
- Writes the sentence they would use in a status update, and has it corrected.

Ask

Your dashboard says twelve done. How many of those can a customer use?

Concepts: durable state, the delivered request, who decides, the record, stages on a board, policy that stays yours.